



Міністерство освіти та науки України
Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»
Факультет інформатики та обчислювальної техніки
Кафедра інформатики і програмної інженерії

Звіт

з дисципліни «Технології розподілених обчислень»
Комп'ютерний практикум №7
«Розробка паралельного алгоритму множення матриць з
використанням MPI-методів колективного обміну повідомленнями та
дослідження його ефективності»

Виконав:

Студент III курсу
гр. ІП-33
Соколов О. В.

Прийняв:

Дифучин А. Ю.
“18” квітня 2026 р.

КОМП'ЮТЕРНИЙ ПРАКТИКУМ №7

Тема: Розробка паралельного алгоритму множення матриць з використанням MPI-методів колективного обміну повідомленнями та дослідження його ефективності

Завдання:

1. Ознайомитись з методами колективного обміну повідомленнями типу «один-до-багатьох», «багато-до-одного», «багато-до-багатьох» (див. лекцію та документацію стандарту MPI).

2. Реалізувати алгоритм паралельного множення матриць з використанням розподілених обчислень в MPI з використанням методів колективного обміну повідомленнями. 40 балів.

3. Дослідити ефективність розподіленого обчислення алгоритму множення матриць при збільшенні розміру матриць та при збільшенні кількості вузлів, на яких здійснюється запуск програми. Порівняйте ефективність алгоритму при використанні методів обміну повідомленнями «один-до-одного», «один-до-багатьох», «багато-до-одного», «багато-до-багатьох». 60 балів.

ВИКОНАННЯ

Завдання 1

Потрібно ознайомитись із колективним обміном типів «один-до-багатьох», «багато-до-одного», «багато-до-багатьох» згідно з лекцією та документацією MPI (п. 1 методичних вказівок).

Колективні виклики виконуються узгоджено всіма процесами комунікатора. Point-to-point (MPI_Send / MPI_Recv) до колективів не належать; такий обмін використано в завд. 3 лише для порівняння.

Один-до-багатьох. MPI_Bcast — однаковий буфер від root усім; MPI_Scatter / MPI_Scatterv — розбиття масиву з root по процесах; Scatterv задає нерівні блоки через counts і displs (розподіл рядків A при $N \not\equiv 0 \pmod{P}$).

Багато-до-одного. MPI_Gather / MPI_Gatherv — збір фрагментів на root; для збору C на ранзі 0 використовується Gatherv з тими ж counts / displs.

Багато-до-багатьох. MPI_Allgather / MPI_Allgatherv — однаковий повний результат на всіх; MPI_Alltoall / MPI_Alltoallv — перестановка блоків між процесами. У завд. 3 застосовано Allgatherv, щоб повний C був на кожному процесі.

Таблиця 1.1 — Патерни обміну в контексті $C = AB$ (розподіл по рядках A).

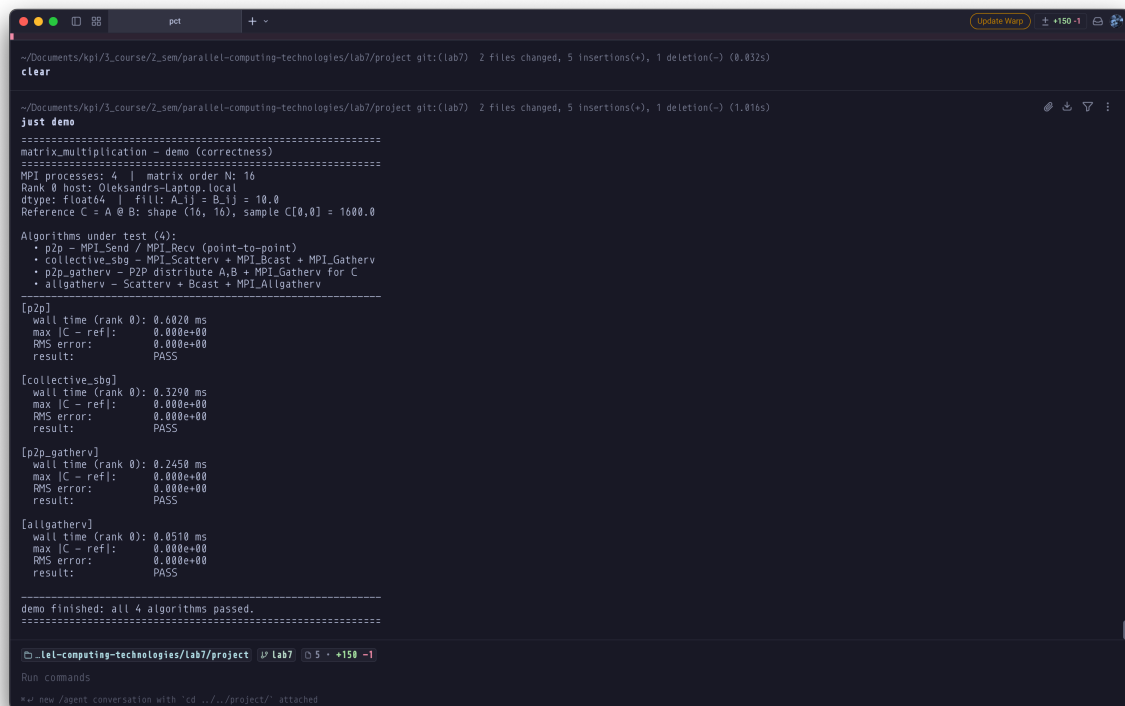
Патерн	Учасники	Примітиви MPI	Роль
Один-до-багатьох	$\text{root} \rightarrow \text{усі}$	Bcast, Scatter, Scatterv	B усім; рядки A по процесах
Багато-до-одного	$\text{усі} \rightarrow \text{root}$	Gather, Gatherv	збір C_{loc} у C на 0
Багато-до-багатьох	$\text{усі} \leftrightarrow \text{усі}$	Allgather, Allgatherv, Alltoall	повний C на всіх або перестановка блоків
Парний обмін	пари рангів	Send, Recv	порівняльний варіант (завд. 3)

Завдання 2

Потрібно реалізувати паралельне множення $C = A \cdot B$ у MPI з колективним обміном (п. 2 методичних вказівок).

На ранзі 0 зберігаються повні A і B . MPI_Scatterv роздає блоки рядків A ; MPI_Bcast — матрицю B усім процесам; локально $C_{\text{loc}} = A_{\text{loc}}B$; MPI_Gatherv збирає фрагменти C на root. Розподіл рядків блоковий; при $N \not\equiv 0 \pmod{P}$ висоти блоків відрізняються не більш ніж на один рядок.

Реалізований варіант позначено як collective_sbg. Коректність перевірена порівнянням з numru на тестових даних під час демонстраційного запуску програми.



```
~/Documents/xpi/2_course/2_sem/parallel-computing-technologies/lab7/project git:(lab7) 2 files changed, 5 insertions(+), 1 deletion(-) (0.832s)
clear

~/Documents/xpi/2_course/2_sem/parallel-computing-technologies/lab7/project git:(lab7) 2 files changed, 5 insertions(+), 1 deletion(-) (1.014s)
just demo

=====
matrix_multiplication - demo (correctness)
=====
MPI processes: 4 | matrix order N: 16
Rank 0 host: Oleksandra-Laptop.local
dtype: float64 | fill: A_ij = B_ij = 10.0
Reference C = A @ B: shape (16, 16), sample C[0,0] = 1600.0

Algorithms under test (4):
• p2p - MPI_Send / MPI_Recv (point-to-point)
• collective_sbg - MPI_Scatterv + MPI_Bcast + MPI_Gatherv
• p2p_gatherv - P2P distribute A,B + MPI_Gatherv for C
• allgatherv - Scatterv + Bcast + MPI_Allgather

=====
[p2p]
wall time (rank 0): 0.6020 ms
max |C - ref|: 0.000e+00
RMS error: 0.000e+00
result: PASS

[collective_sbg]
wall time (rank 0): 0.3290 ms
max |C - ref|: 0.000e+00
RMS error: 0.000e+00
result: PASS

[p2p_gatherv]
wall time (rank 0): 0.2450 ms
max |C - ref|: 0.000e+00
RMS error: 0.000e+00
result: PASS

[allgatherv]
wall time (rank 0): 0.0510 ms
max |C - ref|: 0.000e+00
RMS error: 0.000e+00
result: PASS

=====
demo finished: all 4 algorithms passed.
=====

_lal-computing-technologies/lab7/project lab7 5 · +150 -1
Run commands
* new /agent conversation with 'cd .././project/' attached
```

Рисунок 2.1 — Приклад виводу програми після успішного обчислення

Завдання 3

Потрібно дослідити ефективність при зміні розміру матриць і кількості процесів і порівняти обмін «один-до-одного», «один-до-багатьох», «багато-до-одного», «багато-до-багатьох».

Код	Схема обміну
p2p	Send / Recv для A, B, C
collective_sbg	Scatterv + Bcast + Gatherv (завд. 2)
p2p_gatherv	роздача P2P, збір C — Gatherv
allgatherv	як collective_sbg, збір C — Allgatherv

Вимірюється на ранзі 0 час між MPI_Barrier до й після фази комунікації та локального множення (MPI.Wtime). Для кожної пари (N, P) у таблицю заноситься мінімум із 5 прогонів. Запуск здійснювався через mpirun з окремим виконуваним модулем бенчмарку; режим і розмір матриці задавалися змінними середовища.

Квадратні матриці $N \times N$, $N \in \{100, 150, \dots, 1000\}$ (крок 50), $P \in \{2, 3, \dots, 8\}$. Усі отримані значення зведено в таблиці нижче та у графіки (рис. 3.1–3.6).

Таблиця 3.1 — Приклад: $N = 1000$, $P = 4$ (усі режими).

Варіант	N	P	Час, с
p2p	1000	4	0,008838
collective_sbg	1000	4	0,025622
p2p_gatherv	1000	4	0,016717
allgatherv	1000	4	0,010757

Таблиця 3.2 — Режим завд. 2 (collective_sbg), $N = 1000$.

P	2	3	4	5	6	7	8
Час, с	0,00813	0,01689	0,02562	0,0289	0,02821	0,0348	0,06783

Wall time (s), best of repeats — darker = slower

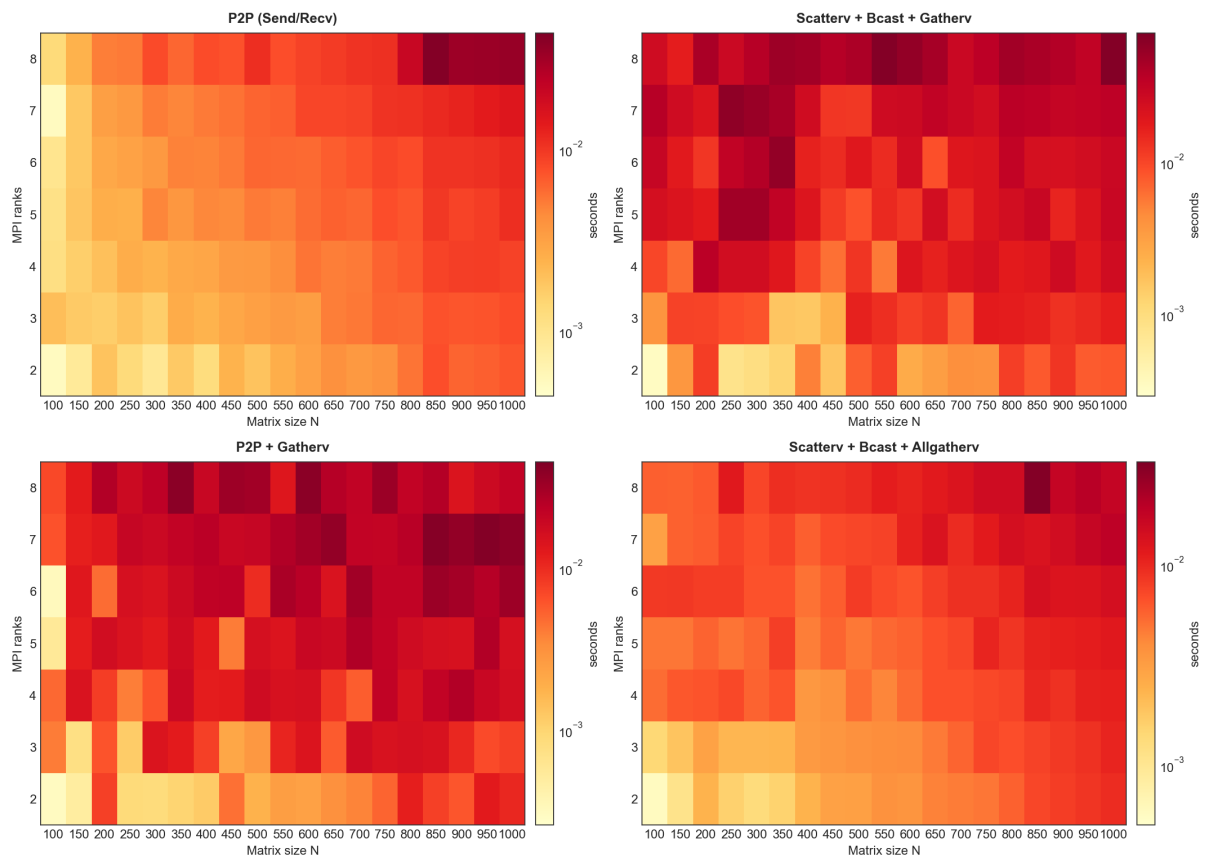


Рисунок 3.1 — Теплокарта $\log_{10}$ часу для чотирьох режимів

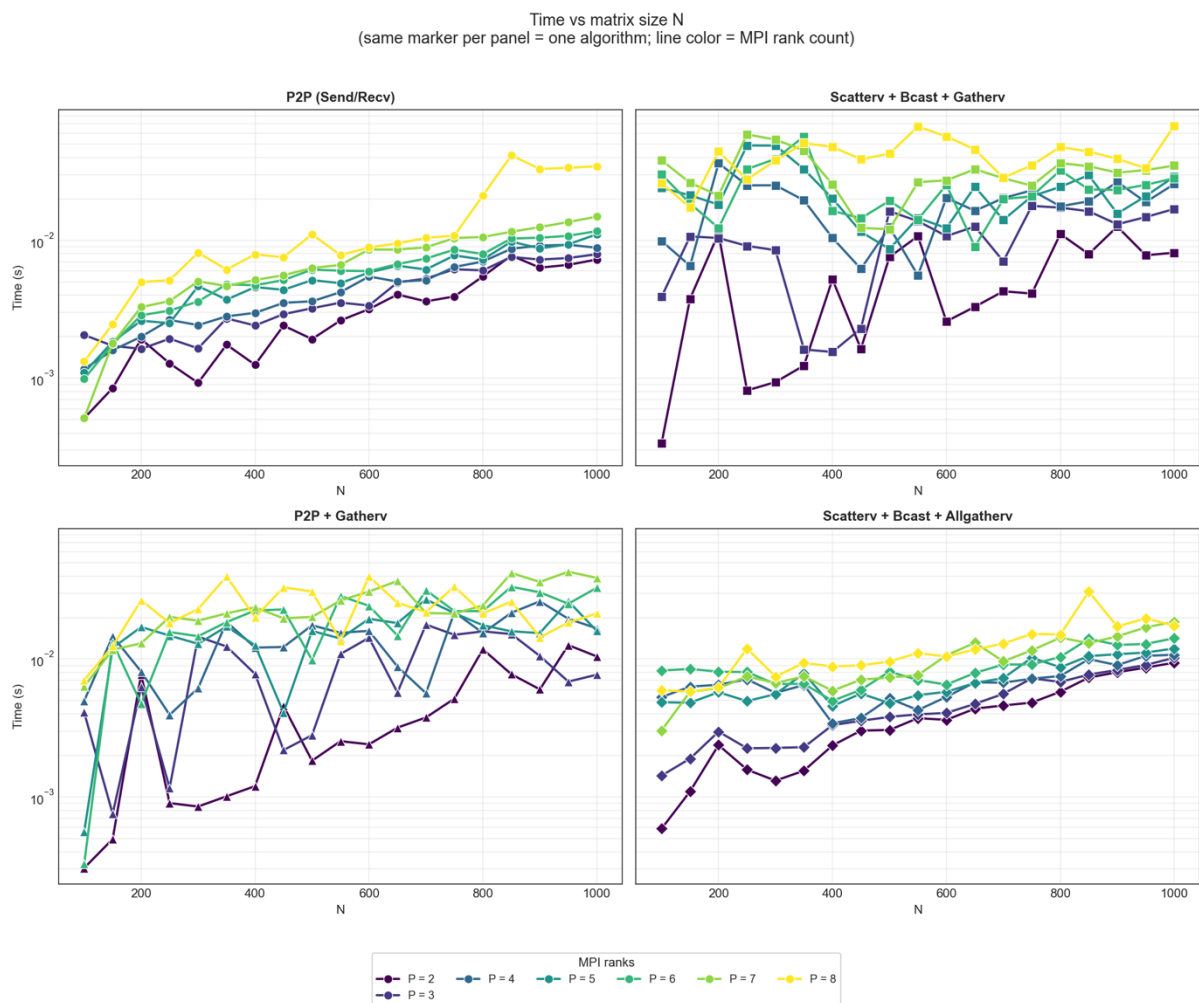


Рисунок 3.2 — Час від N при фіксованих P

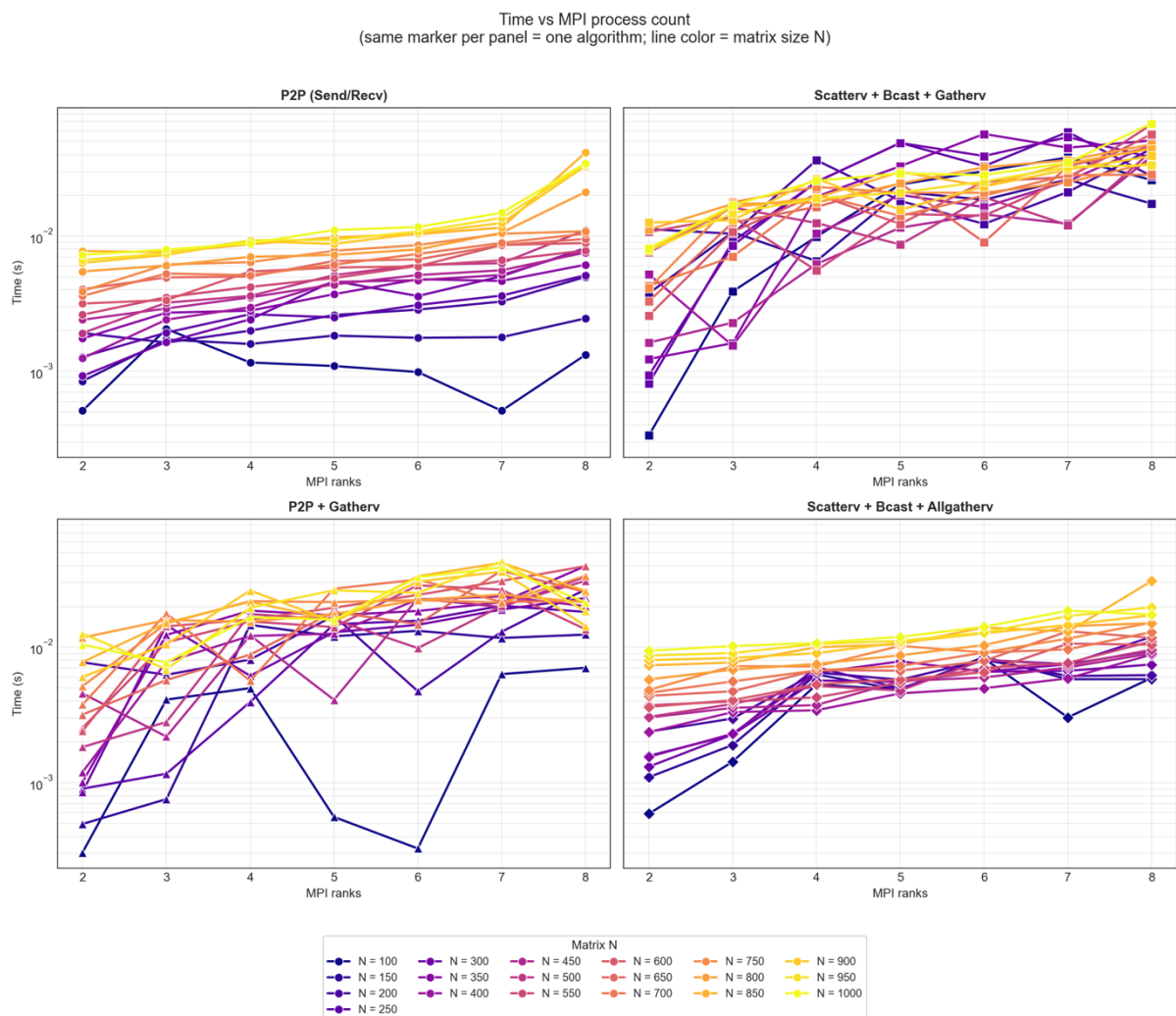


Рисунок 3.3 — Час від P при фіксованих N

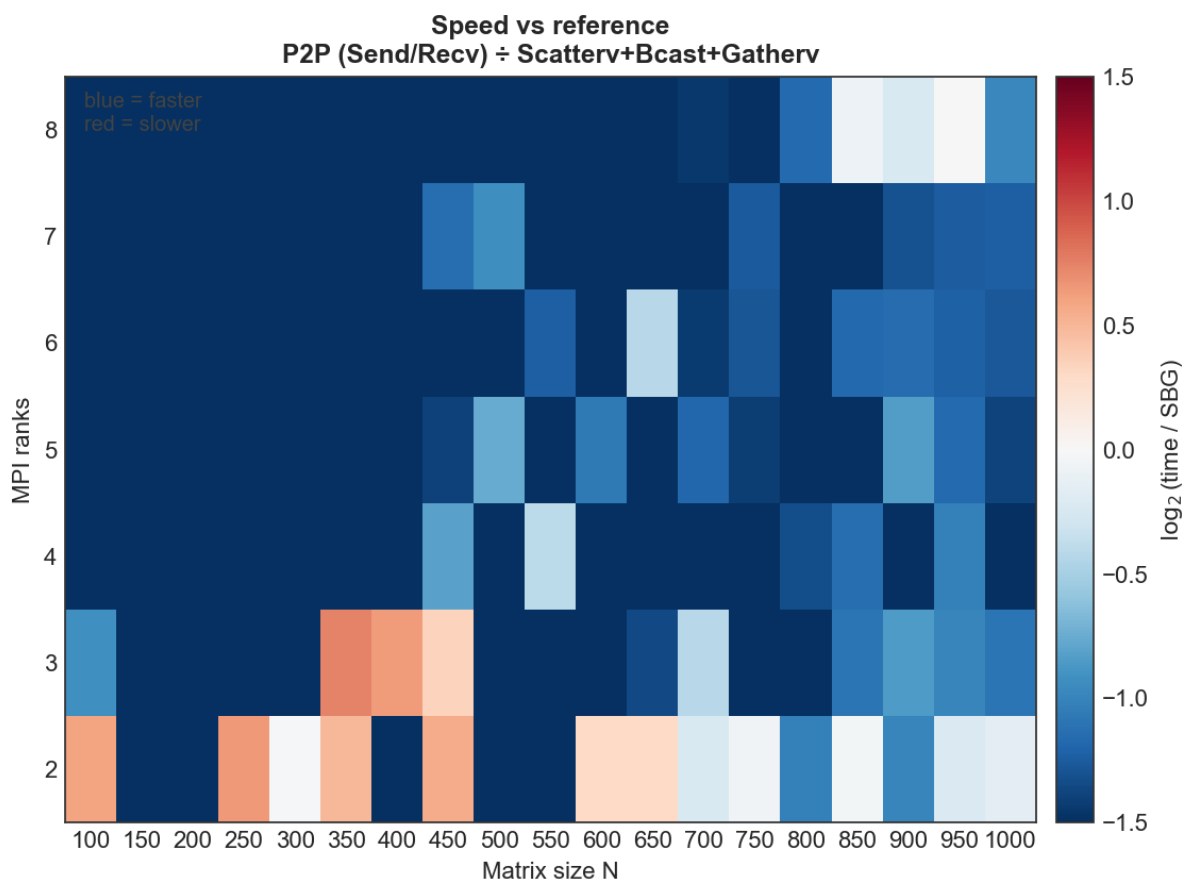


Рисунок 3.4 — Відношення часу до collective_sbg, режим p2p, $\log_2$

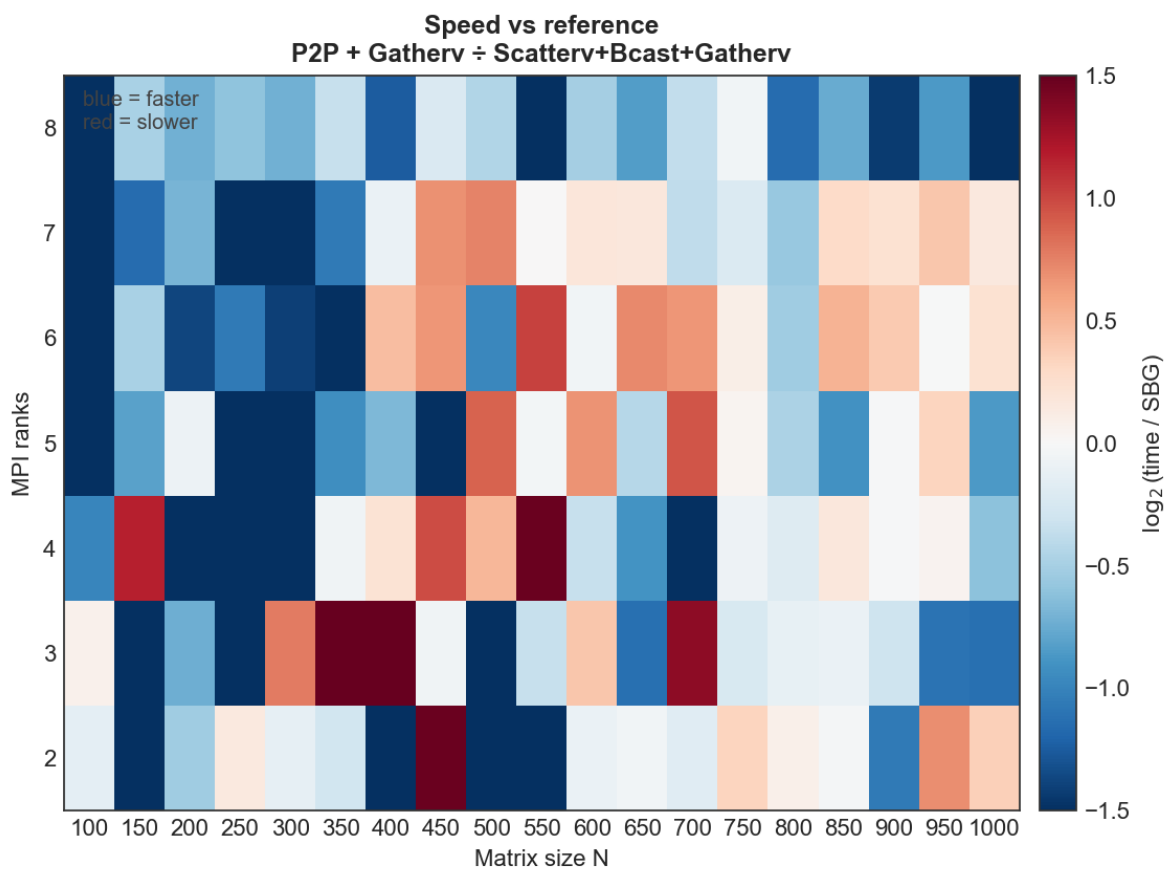


Рисунок 3.5 — Те саме для p2p_gatherv

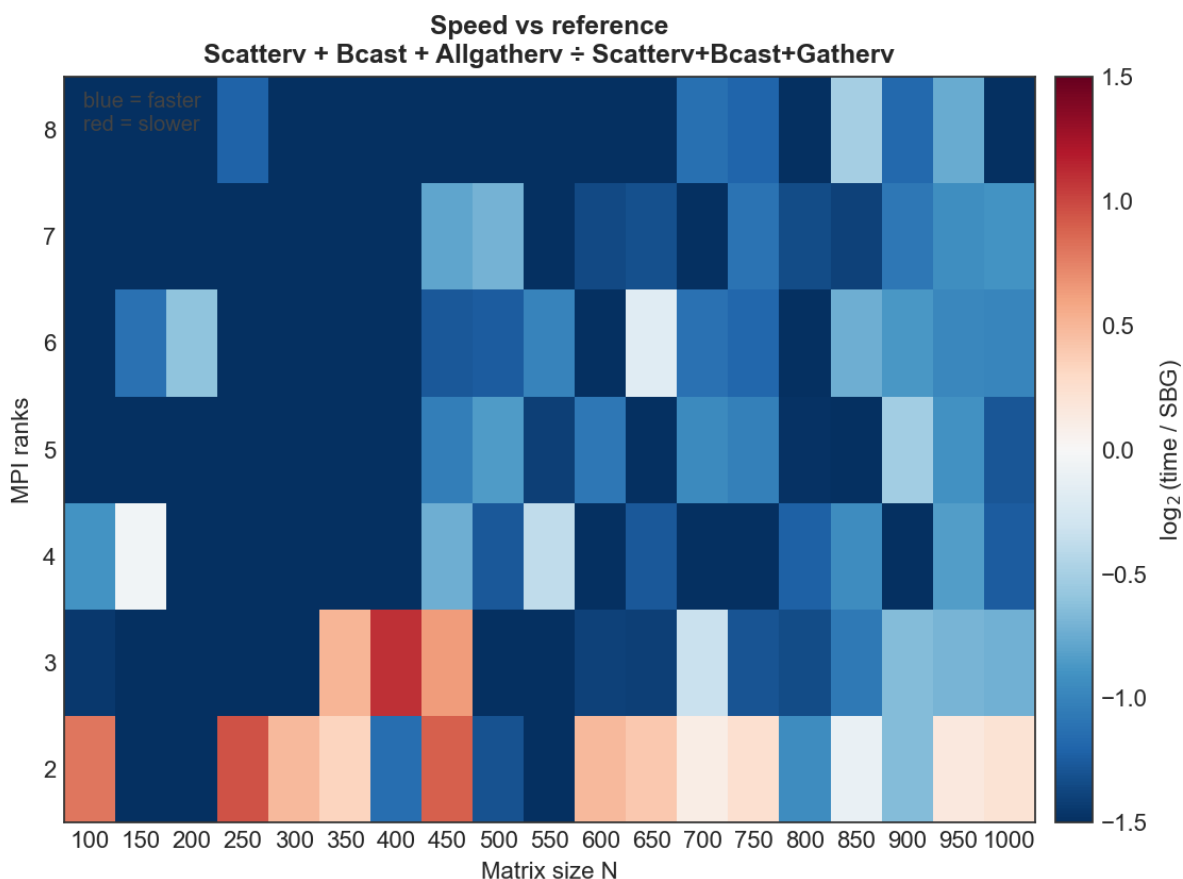


Рисунок 3.6 — Те саме для allgatherv

На рис. 3.1–3.3 час зростає з N ; залежність від P при фіксованому N не дає лінійного прискорення через накладні витрати колективів і передачу B . Для малих N спостерігається розкид точок (шум ОС, мінімум із коротких прогонів). На рис. 3.4–3.6 видно відхилення від `collective_sbg`: `allgatherv` зазвичай повільніший через збір повного C на всіх; `p2p` і `p2p_gatherv` залежно від (N, P) можуть наближатися до еталону або відходити від нього — зокрема через реалізацію MPI та кількість парних передач.

Висновок

У межах практикуму №7 опрацьовано теоретичні основи колективів MPI, реалізовано паралельне множення матриць з Scatterv + Bcast + Gatherv, додано режими для порівняння та виконано експеримент.

Код програми

Посилання

на

GitHub

репозиторій:

<https://github.com/sokolovgit/parallel-computing-technologies>

```
=== ./matrix_multiplication/__init__.py ===
"""Parallel matrix multiply algorithm implementations (MPI + numpy)."""

from matrix_multiplication.base import MatrixMultiplyBase
from matrix_multiplication.registry import ALGOS, iter_algos

__all__ = [
    "ALGOS",
    "MatrixMultiplyBase",
    "iter_algos",
]

=== ./matrix_multiplication/base.py ===
"""Abstract base for parallel C = A @ B implementations."""

from __future__ import annotations

from abc import ABC, abstractmethod

import numpy as np
from mpi4py import MPI

class MatrixMultiplyBase(ABC):
    """Shared contract: full C on rank 0 only; other ranks return None."""

    @property
    @abstractmethod
    def name(self) -> str:
        """Registry key (e.g. ``p2p``)."""

    @abstractmethod
    def multiply(
        self,
        comm: MPI.Comm,
        a_on_root: np.ndarray | None,
        b_on_root: np.ndarray | None,
    ) -> np.ndarray | None:
        """Compute C = A @ B in parallel."""

    @staticmethod
    def serial_gemm(a: np.ndarray, b: np.ndarray) -> np.ndarray:
        return a @ b

    @staticmethod
    def bcast_int_n(comm: MPI.Comm, rank: int, n: int | None) -> int:
        """Broadcast matrix order N from root as int32 (all ranks must
        call)."""
        buf = np.zeros(1, dtype=np.int32)
        if rank == 0:
            assert n is not None
            buf[0] = n
```

```

        comm.Bcast(buf, root=0)
        return int(buf[0])

=== ./matrix_multiplication/implementations/__init__.py ===
"""Concrete matrix-multiply algorithms (MPI communication patterns)."""

from matrix_multiplication.implementations.allgatherv import
CollectiveAllgatherv
from matrix_multiplication.implementations.collective_sbg import
CollectiveScatterBcastGather
from matrix_multiplication.implementations.p2p import P2PMatrixMultiply
from matrix_multiplication.implementations.p2p_gatherv import
P2PDistributeGatherv

__all__ = [
    "CollectiveAllgatherv",
    "CollectiveScatterBcastGather",
    "P2PDistributeGatherv",
    "P2PMatrixMultiply",
]

=== ./matrix_multiplication/implementations/allgatherv.py ===
"""Scatterv + Bcast + Allgatherv."""

from __future__ import annotations

import numpy as np
from mpi4py import MPI

from matrix_multiplication.base import MatrixMultiplyBase
from matrix_multiplication.partition import row_counts,
scatter_counts_and_displs

class CollectiveAllgatherv(MatrixMultiplyBase):
    @property
    def name(self) -> str:
        return "allgatherv"

    def multiply(
        self,
        comm: MPI.Comm,
        a_on_root: np.ndarray | None,
        b_on_root: np.ndarray | None,
    ) -> np.ndarray | None:
        rank = comm.Get_rank()
        size = comm.Get_size()

        n_root: int | None = None
        if rank == 0:
            assert a_on_root is not None and b_on_root is not None
            a = np.ascontiguousarray(a_on_root, dtype=np.float64)
            b = np.ascontiguousarray(b_on_root, dtype=np.float64)
            n_root = a.shape[0]
            n = self.bcast_int_n(comm, rank, n_root)

        if size == 1:
            if rank == 0:

```

```

        assert a_on_root is not None and b_on_root is not None
        a = np.ascontiguousarray(a_on_root, dtype=np.float64)
        b = np.ascontiguousarray(b_on_root, dtype=np.float64)
        return self.serial_gemm(a, b)
    return None

    counts = row_counts(n, size)
    scounts, displs = scatter_counts_and_displs(n, size)

    my_rows = counts[rank]
    recv_a = np.empty(max(my_rows * n, 0), dtype=np.float64)
    if rank == 0:
        flat_a = np.ascontiguousarray(a.ravel())
        comm.Scatterv([flat_a, scounts, displs, MPI.DOUBLE], recv_a,
root=0)
    else:
        comm.Scatterv(None, recv_a, root=0)

    b_loc = np.empty((n, n), dtype=np.float64)
    if rank == 0:
        b_loc[:] = b
    comm.Bcast(b_loc, root=0)

    if my_rows > 0:
        a_loc = recv_a[: my_rows * n].reshape(my_rows, n)
        c_loc = (a_loc @ b_loc).ravel()
    else:
        c_loc = np.empty(0, dtype=np.float64)

    recv_c = np.empty(n * n, dtype=np.float64)
    comm.Allgatherv(c_loc, [recv_c, scounts, displs, MPI.DOUBLE])

    return recv_c.reshape(n, n) if rank == 0 else None

=== ./matrix_multiplication/implementations/collective_sbg.py ===
"""Scatterv + Bcast + Gatherv (task 2 main collective variant)."""

from __future__ import annotations

import numpy as np
from mpi4py import MPI

from matrix_multiplication.base import MatrixMultiplyBase
from matrix_multiplication.partition import row_counts,
scatter_counts_and_displs

class CollectiveScatterBcastGather(MatrixMultiplyBase):
    @property
    def name(self) -> str:
        return "collective_sbg"

    def multiply(
        self,
        comm: MPI.Comm,
        a_on_root: np.ndarray | None,
        b_on_root: np.ndarray | None,
    ) -> np.ndarray | None:

```

```

rank = comm.Get_rank()
size = comm.Get_size()

n_root: int | None = None
if rank == 0:
    assert a_on_root is not None and b_on_root is not None
    a = np.ascontiguousarray(a_on_root, dtype=np.float64)
    b = np.ascontiguousarray(b_on_root, dtype=np.float64)
    n_root = a.shape[0]
    assert a.shape[0] == a.shape[1] == b.shape[0] == b.shape[1]
    n = self.bcast_int_n(comm, rank, n_root)

if size == 1:
    if rank == 0:
        assert a_on_root is not None and b_on_root is not None
        a = np.ascontiguousarray(a_on_root, dtype=np.float64)
        b = np.ascontiguousarray(b_on_root, dtype=np.float64)
        return self.serial_gemm(a, b)
    return None

counts = row_counts(n, size)
scounts, displs = scatter_counts_and_displs(n, size)

my_rows = counts[rank]
recv_a = np.empty(max(my_rows * n, 0), dtype=np.float64)
if rank == 0:
    flat_a = np.ascontiguousarray(a.ravel())
    comm.Scatterv([flat_a, scounts, displs, MPI.DOUBLE], recv_a,
root=0)
else:
    comm.Scatterv(None, recv_a, root=0)

b_loc = np.empty((n, n), dtype=np.float64)
if rank == 0:
    b_loc[:] = b
comm.Bcast(b_loc, root=0)

if my_rows > 0:
    a_loc = recv_a[: my_rows * n].reshape(my_rows, n)
    c_loc = (a_loc @ b_loc).ravel()
else:
    c_loc = np.empty(0, dtype=np.float64)

recv_c = np.empty(n * n, dtype=np.float64) if rank == 0 else None

if rank == 0:
    assert recv_c is not None
    comm.Gatherv(c_loc, [recv_c, scounts, displs, MPI.DOUBLE],
root=0)
    return recv_c.reshape(n, n)
comm.Gatherv(c_loc, None, root=0)
return None

=== ./matrix_multiplication/implementations/p2p.py ===
"""One-to-one: MPI_Send / MPI_Recv for A strips, B, and C strips."""

from __future__ import annotations

```



```

import numpy as np
from mpi4py import MPI

from matrix_multiplication.base import MatrixMultiplyBase
from matrix_multiplication.partition import row_counts

class P2PMatrixMultiply(MatrixMultiplyBase):
    @property
    def name(self) -> str:
        return "p2p"

    def multiply(
        self,
        comm: MPI.Comm,
        a_on_root: np.ndarray | None,
        b_on_root: np.ndarray | None,
    ) -> np.ndarray | None:
        rank = comm.Get_rank()
        size = comm.Get_size()

        if rank == 0:
            assert a_on_root is not None and b_on_root is not None
            a = np.ascontiguousarray(a_on_root, dtype=np.float64)
            b = np.ascontiguousarray(b_on_root, dtype=np.float64)
            n = a.shape[0]
            assert a.shape == (n, n) and b.shape == (n, n)
        else:
            a = np.empty((0, 0), dtype=np.float64)
            b = np.empty((0, 0), dtype=np.float64)
            n = 0

        if size == 1:
            if rank == 0:
                assert a_on_root is not None and b_on_root is not None
                a = np.ascontiguousarray(a_on_root, dtype=np.float64)
                b = np.ascontiguousarray(b_on_root, dtype=np.float64)
                return self.serial_gemm(a, b)
            return None

        if rank != 0:
            n_arr = np.empty(1, dtype=np.int32)
            comm.Recv(n_arr, source=0, tag=8)
            n = int(n_arr[0])

        counts = row_counts(n, size)

        if rank == 0:
            c_full = np.empty((n, n), dtype=np.float64)
            offset = counts[0]
            for dest in range(1, size):
                rows = counts[dest]
                comm.Send(np.int32(n), dest=dest, tag=8)
                comm.Send(np.int32(offset), dest=dest, tag=10)
                comm.Send(np.int32(rows), dest=dest, tag=11)
                if rows > 0:
                    comm.Send(a[offset : offset + rows, :], dest=dest,
tag=12)

```

```

        comm.Send(b, dest=dest, tag=13)
        offset += rows

    r0 = counts[0]
    if r0 > 0:
        c_full[0:r0, :] = a[0:r0, :] @ b

    for src in range(1, size):
        rows = counts[src]
        if rows > 0:
            off = sum(counts[:src])
            comm.Recv(c_full[off : off + rows, :], source=src,
tag=14)

    return c_full

off = np.empty(1, dtype=np.int32)
rows_arr = np.empty(1, dtype=np.int32)
comm.Recv(off, source=0, tag=10)
comm.Recv(rows_arr, source=0, tag=11)
rows = int(rows_arr[0])
if rows > 0:
    a_loc = np.empty((rows, n), dtype=np.float64)
    comm.Recv(a_loc, source=0, tag=12)
    b_loc = np.empty((n, n), dtype=np.float64)
    comm.Recv(b_loc, source=0, tag=13)
    c_loc = a_loc @ b_loc
    comm.Send(c_loc, dest=0, tag=14)
return None

```

```

=== ./matrix_multiplication/implementations/p2p_gatherv.py ===
"""P2P distribution; Gatherv for C."""

```

```

from __future__ import annotations

```

```

import numpy as np
from mpi4py import MPI

```

```

from matrix_multiplication.base import MatrixMultiplyBase
from matrix_multiplication.partition import row_counts,
scatter_counts_and_displs

```

```

class P2PDistributeGatherv(MatrixMultiplyBase):
    @property
    def name(self) -> str:
        return "p2p_gatherv"

    def multiply(
        self,
        comm: MPI.Comm,
        a_on_root: np.ndarray | None,
        b_on_root: np.ndarray | None,
    ) -> np.ndarray | None:
        rank = comm.Get_rank()
        size = comm.Get_size()

        if rank == 0:
            assert a_on_root is not None and b_on_root is not None

```

```

        a = np.ascontiguousarray(a_on_root, dtype=np.float64)
        b = np.ascontiguousarray(b_on_root, dtype=np.float64)
        n = a.shape[0]
    else:
        n_arr = np.empty(1, dtype=np.int32)
        comm.Recv(n_arr, source=0, tag=8)
        n = int(n_arr[0])
        a = np.empty((0, 0), dtype=np.float64)
        b = np.empty((0, 0), dtype=np.float64)

    if size == 1:
        if rank == 0:
            assert a_on_root is not None and b_on_root is not None
            a = np.ascontiguousarray(a_on_root, dtype=np.float64)
            b = np.ascontiguousarray(b_on_root, dtype=np.float64)
            return self.serial_gemm(a, b)
        return None

    counts = row_counts(n, size)
    scounts, displs = scatter_counts_and_displs(n, size)

    if rank == 0:
        offset = counts[0]
        for dest in range(1, size):
            rows = counts[dest]
            comm.Send(np.int32(n), dest=dest, tag=8)
            comm.Send(np.int32(offset), dest=dest, tag=10)
            comm.Send(np.int32(rows), dest=dest, tag=11)
            if rows > 0:
                comm.Send(a[offset : offset + rows, :], dest=dest,
tag=12)
                comm.Send(b, dest=dest, tag=13)
            offset += rows

    my_rows = counts[rank]
    if rank == 0:
        if my_rows > 0:
            a_loc = a[0:my_rows, :].copy()
        else:
            a_loc = np.empty((0, n), dtype=np.float64)
        b_loc = b.copy()
    else:
        off = np.empty(1, dtype=np.int32)
        rows_arr = np.empty(1, dtype=np.int32)
        comm.Recv(off, source=0, tag=10)
        comm.Recv(rows_arr, source=0, tag=11)
        rows = int(rows_arr[0])
        if rows > 0:
            a_loc = np.empty((rows, n), dtype=np.float64)
            comm.Recv(a_loc, source=0, tag=12)
            b_loc = np.empty((n, n), dtype=np.float64)
            comm.Recv(b_loc, source=0, tag=13)
        else:
            a_loc = np.empty((0, n), dtype=np.float64)
            b_loc = np.empty((0, 0), dtype=np.float64)

    if my_rows > 0:
        c_loc = (a_loc @ b_loc).ravel()

```

```

        else:
            c_loc = np.empty(0, dtype=np.float64)

            recv_c = np.empty(n * n, dtype=np.float64) if rank == 0 else None
            if rank == 0:
                comm.Gatherv(c_loc, [recv_c, scounts, displs, MPI.DOUBLE],
root=0)
                assert recv_c is not None
                return recv_c.reshape(n, n)
            comm.Gatherv(c_loc, None, root=0)
            return None

=== ./matrix_multiplication/partition.py ===
from __future__ import annotations

def row_counts(n_rows: int, n_procs: int) -> list[int]:
    """Rows per rank (block distribution)."""
    if n_procs <= 0:
        raise ValueError("n_procs must be positive")
    base = n_rows // n_procs
    rem = n_rows % n_procs
    return [base + (1 if i < rem else 0) for i in range(n_procs)]

def scatter_counts_and_displs(n: int, size: int) -> tuple[list[int],
list[int]]:
    """Element counts and displacements for row-major flattened A (N*N
elements)."""
    rows = row_counts(n, size)
    counts = [r * n for r in rows]
    displs: list[int] = []
    acc = 0
    for c in counts:
        displs.append(acc)
        acc += c
    return counts, displs

=== ./matrix_multiplication/registry.py ===
from __future__ import annotations

from collections.abc import Iterator

from matrix_multiplication.base import MatrixMultiplyBase
from matrix_multiplication.implementations import (
    CollectiveAllgatherv,
    CollectiveScatterBcastGather,
    P2PDistributeGatherv,
    P2PMatrixMultiply,
)

ALGOS: dict[str, MatrixMultiplyBase] = {
    "p2p": P2PMatrixMultiply(),
    "collective_sbg": CollectiveScatterBcastGather(),
    "p2p_gatherv": P2PDistributeGatherv(),
    "allgatherv": CollectiveAllgatherv(),
}

```

```
def iter_algos() -> Iterator[tuple[str, MatrixMultiplyBase]]:
    for k, v in ALGOS.items():
        yield k, v
```